HUAWEI IRELAND RESEARCH CENTRE

UCD DUBLIN University College Dublin

IDs That Mean Something

A Hands-On Tutorial on Semantic IDs for Recommender Systems

Roger Zhe Li, Aghiles Salah, Mete Sertkan, Changhong Jin, Yingjie Niu, Ruihai Dong, Barry Smyth

CIKM 26 | Rome, Italy

Nov 7, 2026



Presenters & Instructors



Roger Zhe Li

Huawei Ireland Research Center



Aghiles Salah

Huawei Ireland Research Center



Mete Sertkan

Huawei Ireland Research Center



Changhong Jin

University College Dublin



Yingjie Niu

Huawei Ireland Research Center



Ruihai Dong

University College Dublin



Barry Smyth

University College Dublin

Supplementary Materials

Tutorial slides, code, and our curated reading list are available on the tutorial website:



https://semantic-id-tutorial.github.io/

Overview & Agenda

Part I: Foundations **85m**

- Introduction to recommender systems, raw IDs [25m]
- Semantic IDs: introduction and overview [25m]
- Leveraging semantic IDs in recommender systems [35m]

Part II: Applications **95m**

- Hands-on: building and using semantic IDs [35m]
- Multimodality and collaborative signals [30m]
- Cold-start mitigation and cross-domain transfer [30m]

Part III: Perspectives **60m**

- Hands-on: cross-domain usage of Semantic IDs [30m]
- Future directions and open challenges [15m]
- Q&A and open discussion [15m]

14:00–18:30 · 4 hours of content + 30-minute tea break (16:00–16:30)

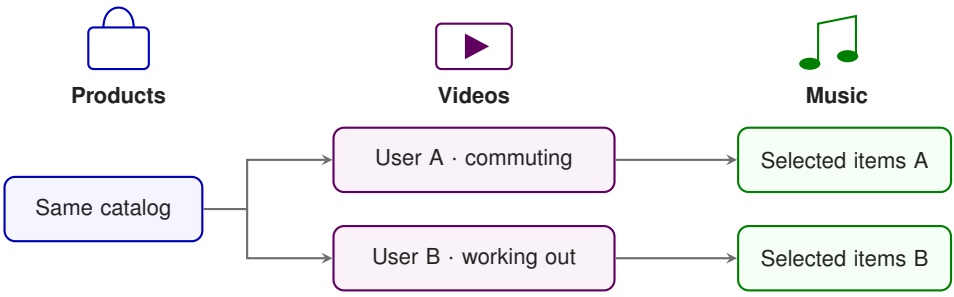
Part 1

Foundations of Recommender Systems & Semantic IDs

Foundations of Recommender Systems

What Is a Recommender System?

Select and prioritize items that may be useful or relevant to a user in a given context.



Users, items and context define the recommendation problem.

What Should We Recommend?

An item the user has viewed



Ultraboost 5

A · Core Black

?

Which item fits?



Ultraboost 5

B · Cloud White



Adizero EVO SL

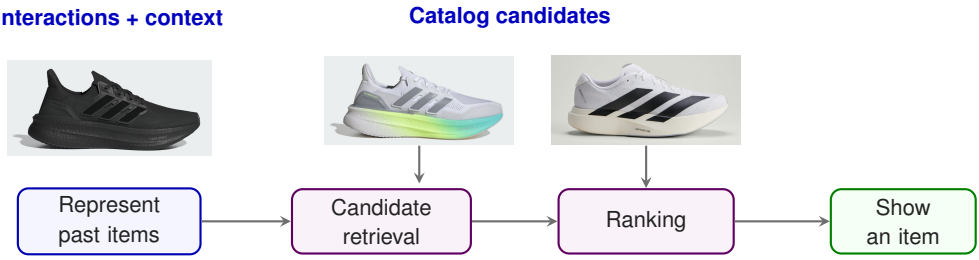
C · Cloud White

History + context

Same brand and category; different products.
Which one fits this user, in this context?

Recommendation chooses an item using evidence about users, items and context.

Where Do Item Representations Enter?

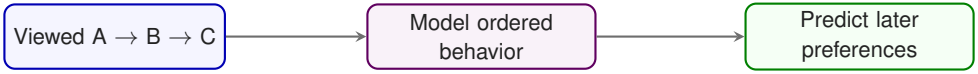


Both past interactions and candidate scoring need a representation of each item.

Before choosing the model, we need a way to represent its items.

Sequential and Generative Recommendation

Sequential: use the order of interactions



Generative: produce an item identifier



Sequential modeling describes the context; generation describes the output interface. They can be combined.

Raw IDs Learn Valuable Behavioral Signals



A: Ultraboost 5

Raw ID
1042

Trainable embedding table

1041
1042
1043

Interactions train the selected row.
ID numbers themselves
have no semantic distance.

Recommendation model

Preferences
Substitutes and
complements

Raw IDs learn valuable behavioral signals.
But does this solve every recommendation problem?

A New Item Has No Interaction History Yet

Established item A



Ultraboost 5 · Raw ID 1042

Many interactions
Repeated embedding updates

New item B



Ultraboost 5 · Raw ID 2087

No interactions yet
Image and description
already available

Teaching scenario: the white colorway is newly listed in this catalog.

A new ID cannot learn a well-supported item-specific vector from its own interactions yet.

An ID Does Not Carry Its Meaning Across Domains

Store X

Local ID: **1042**
Local interactions

↓

Embedding table X



The same physical product

Store Y

Local ID: **8831**
Different interactions

↓

Embedding table Y

- - - - No automatic alignment - - - -

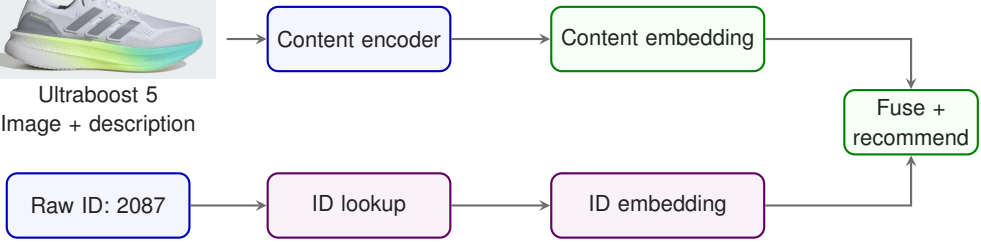
Transfer needs an additional bridge: entity matching, shared features or learned alignment.

Independent raw ID tables do not establish correspondence or transfer knowledge by themselves.

Content Features Are a Natural Next Step



Ultraboost 5
Image + description



The same representation pathways can be used for history items and candidates.

Content features are already a usable recommendation solution, and can complement raw IDs.

Content Features: Encoding, Storage and Serving



Image + description

Content encoder

Vector cache

1042

2087

3091

Offline: compute and refresh

Online: fetch and use

History: [1042, 3091, . . .]
Candidate: 2087

Fetch vectors
Fuse + recommend

Encoding updates
New or changed content

Storage and movement
Items $\times$ vector dimension

Precomputation avoids repeated encoding; serving and maintenance costs still matter.

IDs That Mean Something: semantic codes let related items share parameters.
This can support new items before they accumulate their own interactions.

Our Scope and Perspective

What we cover

Construct and enrich IDs
Quantization and richer item signals.

Generate and rank
SID targets and SID input features.

Build and evaluate
Hands-on, cold start and cross-domain use.

Not covered in depth

Comprehensive model surveys
All RecSys algorithms and LLM architectures.

Paper-specific training recipes
Every specialized loss or training trick.

Production implementation
Detailed deployment engineering.

Our lens: item identity and information sharing.
Complementing model-centered tutorials through construction, use and hands-on evaluation.

What information should an ID carry, what should it share, and how do we know it helps?

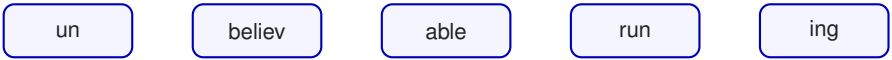
Li et al., *IDs That Mean Something: A Hands-On Tutorial on Semantic IDs for Recommender Systems.*

Introduction to Semantic IDs

How NLP Uses a Reusable Vocabulary



A reusable vocabulary, fixed for this model version

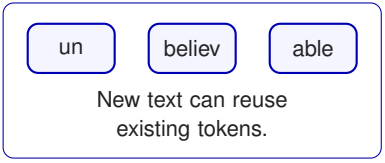


Many words can reuse these entries. Each token indexes a trainable embedding row.

A fixed vocabulary can represent many inputs through reusable pieces.

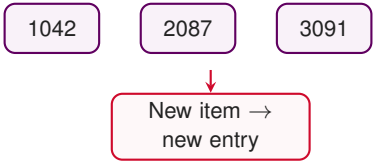
Why Not Give Every Item Its Own Token?

Reusable NLP vocabulary



Vocabulary size is bounded
within a model version.

One token per catalog item



More items mean more entries.
New and rare entries
lack training coverage.

The question is how to share a manageable vocabulary across an expanding catalog.

A First Attempt: Encode the Taxonomy Path



Explicitly defined semantics

Predefined categories determine the meaning.

Accuracy and consistency

Labels can be wrong or differ across stores.

Granularity and relevance

One path can group items with different uses.

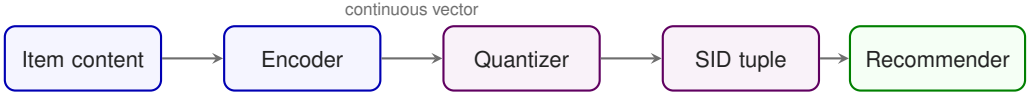
Maintenance and alignment

Categories and store mappings need updates.

Explicit taxonomies enable sharing; their accuracy, consistency and granularity still need validation.

Google Merchant Center, *Google product category [google_product_category].*

From Content Embeddings to Semantic IDs





A: (a_{12}, b_7, c_3)



B: (a_{12}, b_7, c_9)

SID = Semantic ID

A discrete code sequence derived from an item representation.

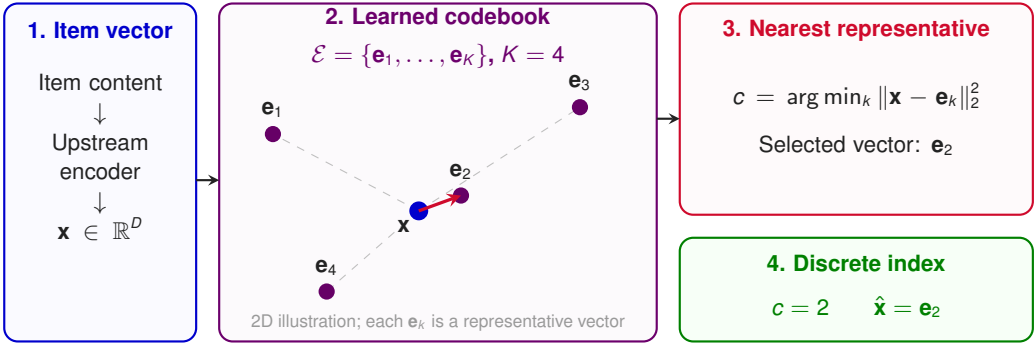
a_{12}, b_7, c_3: entry indices in codebooks 1, 2 and 3.

A and B share the first two codes; the last differs.
These are indices, not vector values or fixed brand/category labels.

Semantic IDs discretize representations; their sharing, quality and costs must be evaluated.

Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

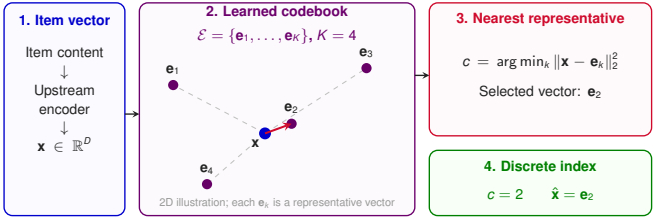
Vector Quantization: Architecture and Vector Discretization



Read the diagram: given an item vector and an already learned codebook, find the nearest representative $\mathbf{e}_k$. Output its index c; use the selected vector to approximate the input.

Gray, *Vector Quantization*, IEEE ASSP Magazine, 1984.

Vector Quantization — Efficiency vs. Scalability



Advantages

- **Compact Representation**: Represents each vector with a single discrete code.
- **Simple Lookup**: Enables efficient storage and retrieval with a finite codebook.

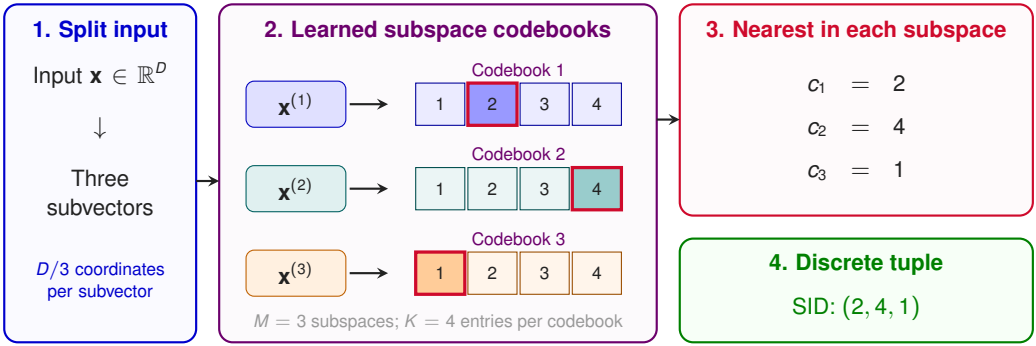
Limitations

- **Limited Capacity**: one K-entry codebook provides at most K distinct indices. Many items can receive the same code.
- **Finer Distinctions**: use a larger codebook or combine multiple codes $\rightarrow$ *Subspace Decomposition*.

Gray, *Vector Quantization*, IEEE ASSP Magazine, 1984.

Product Quantization — Overview

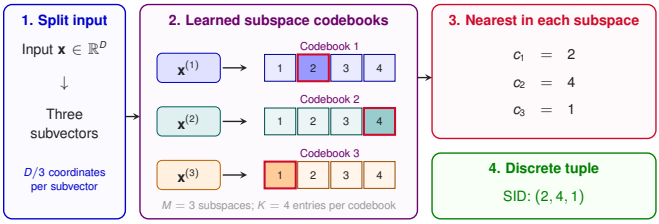
Split a full-vector quantization problem into smaller subspace problems.



$\mathbf{x}^{(m)}$ is a subvector; c_m is a scalar code index. Concatenate the selected representatives to reconstruct $\mathbf{x}$.

Jégou et al., *Product Quantization for Nearest Neighbor Search*, TPAMI 2011.

Product Quantization — Bridging Scalability and Space Optimization



Advances over VQ

- **Combinatorial Capacity**: M codebooks of K entries allow up to K^M tuples; not all need be occupied.
- **Compact Codebooks**: equal subspaces store $MK(D/M) = KD$ scalar values.

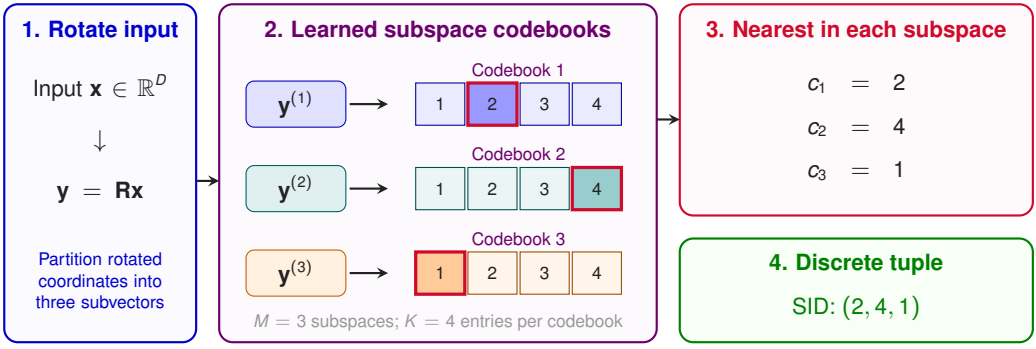
Limitations

- **Naive Axis-Splitting**: Ignores sub-space correlation $\rightarrow$ *Need Space Rotation*.
- **Uneven Approximation**: fixed groups may differ in quantization difficulty.

Jégou et al., *Product Quantization for Nearest Neighbor Search*, TPAMI 2011.

Optimized Product Quantization — Learned Rotation before Splitting

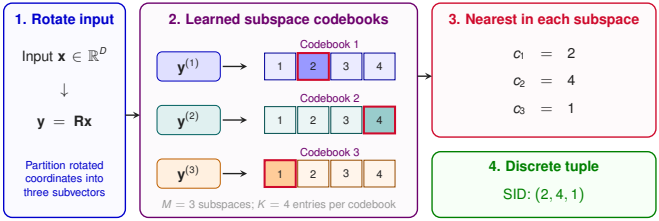
Learn a rotation that makes the subspace partition fit the data better.



$\mathbf{y} = \mathbf{R}\mathbf{x}$, with $\mathbf{R}^{\mathsf{T}}\mathbf{R} = \mathbf{I}$. Quantize subvectors of $\mathbf{y}$; reconstruction applies the inverse rotation.

Ge et al., *Optimized Product Quantization for Approximate Nearest Neighbor Search*, CVPR 2013.

Optimized Product Quantization — Improving the Subspace Decomposition



Advances over PQ

- **Learned Rotation**: an orthogonal matrix **R** mixes coordinates before splitting.
- **Better Fit**: joint optimization can reduce quantization error; independence is not guaranteed.

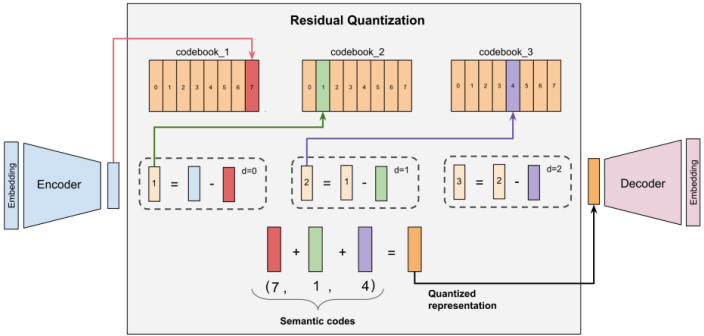
Limitations

- **Still Horizontal**: Rotation only re-orients the split; the output $[A, B, C, D]$ remains a set of *parallel*, unordered codes.
- **No Coarse-to-Fine Structure**: Parallel subspace codes do not directly model residual refinement $\rightarrow$ *Residual Quantization*.

Ge et al., *Optimized Product Quantization for Approximate Nearest Neighbor Search*, CVPR 2013.

Residual Quantization — Overview

Each layer quantizes the residual left by earlier layers.

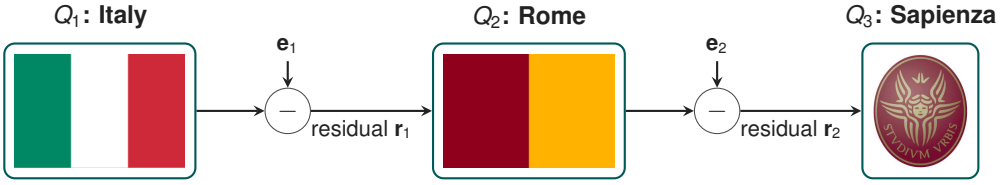


Encoder output **z** starts the residual chain; each codebook selects a correction.

Sum the selected vectors into $\hat{\mathbf{z}}$ for reconstruction; keep their indices as the discrete SID.

Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

Residual Quantization — Mechanism with a Localized Demo



Formulation	Coarse-to-Fine Quantization
$\mathbf{r}_0 = \mathbf{z}$	Input latent: $\mathbf{z} = \mathrm{Encoder}(\mathbf{x})$; start with $\mathbf{z}$
$\mathbf{r}_1 = \mathbf{r}_0 - \mathbf{e}_1$	Layer 1 (Q_1): Capture coarse region → **Italy**
$\mathbf{r}_2 = \mathbf{r}_1 - \mathbf{e}_2$	Layer 2 (Q_2): Quantize intermediary residual → **Rome**
$\hat{\mathbf{z}} = \sum_{\ell=1}^{L} \mathbf{e}_\ell$	Layer 3 (Q_3): Resolve fine residual → **Sapienza**

L: quantization layers. Geographic labels illustrate refinement, not guaranteed learned meanings.

Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

Residual Quantization — Why It Is a Milestone



Why It Is a Milestone

- **Residual Refinement**: later layers correct what earlier layers leave unexplained.
- **Compact Capacity**: $K = 256$, $L = 4$: up to 4.29×10^9 tuples. With $D = 128$, FP32 codebooks occupy just 0.5 MiB.

Limitations

- **Sequential Construction**: L dependent layers; $O(LKD)$ exhaustive assignment.
- **Finite Precision**: a finite residual chain leaves quantization error; different items can still share a code.

Capacity does not ensure unique assignments or recommendation quality. Quantizer depth and downstream autoregressive decoding are separate costs.

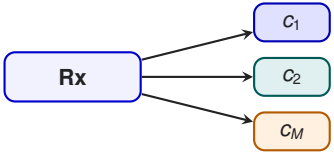
Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

Long Semantic IDs — Two Paths toward Longer Codes

Shared Goal: Increase parallel subspaces, or deepen residual refinement.

Parallel Construction

RPG



- Quantize more OPQ subspaces **in parallel**.
- Increase capacity without extending a sequential chain.
- Each code describes a different subspace.

Structured Expansion

LSIG



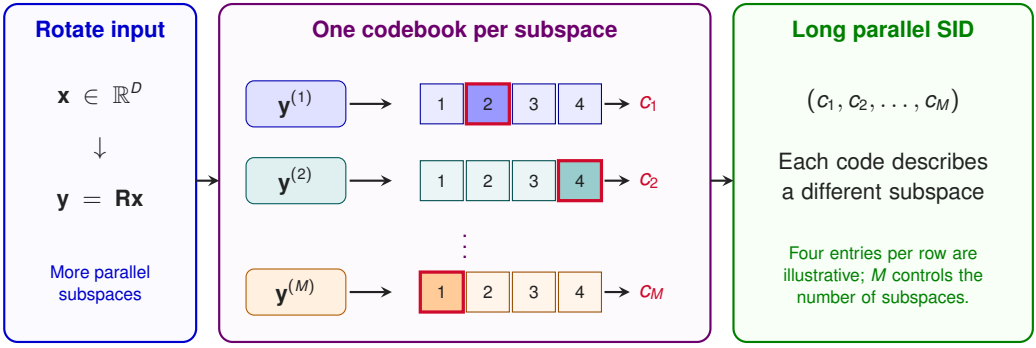
- Extend residual quantization with **more layers**.
- Each layer quantizes the remaining residual.
- Preserve semantic organization during ID expansion.

Same goal, different strategies: RPG uses **parallel subspace codes**, while LSIG illustrates **longer residual hierarchies**.

Hou et al., *Generating Long Semantic IDs in Parallel for Recommendation*, KDD 2025.

Long Semantic IDs — Scaling Capacity via Parallel Generation

More OPQ subspaces produce a longer tuple of parallel codes.

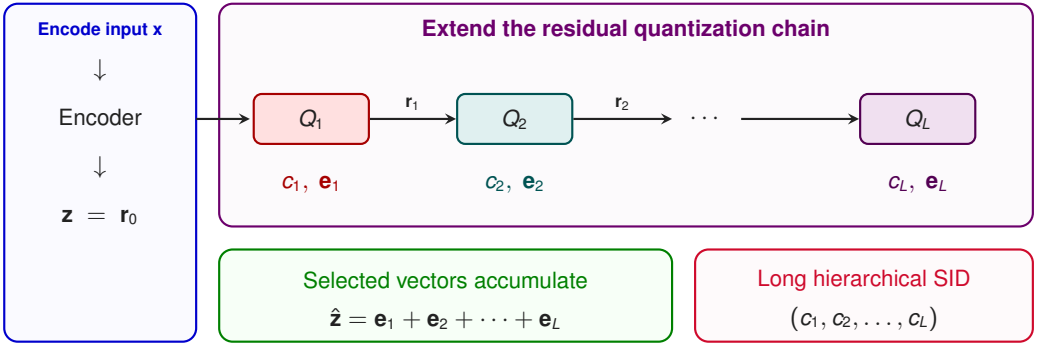


Each branch quantizes a different subspace; increasing M adds parallel code positions. RPG pairs these IDs with parallel token prediction. Here we focus on construction; training and decoding mechanisms are outside this section.

Hou et al., *Generating Long Semantic IDs in Parallel for Recommendation*, KDD 2025.

Long Semantic IDs — Extending the Residual Hierarchy

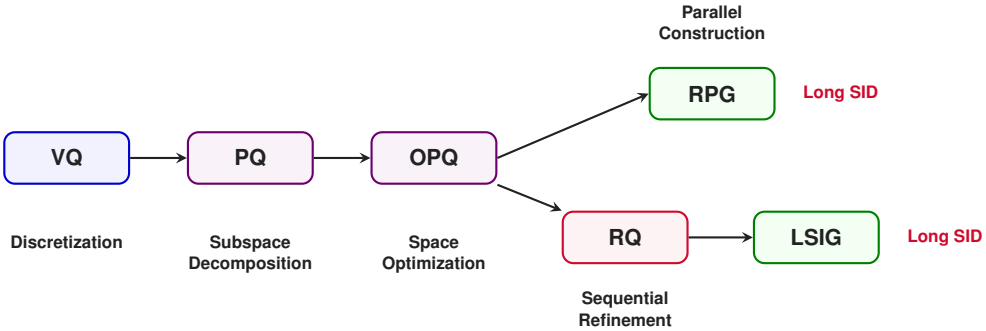
More residual layers produce a longer hierarchical SID.



At layer ℓ, selected vector $\mathbf{e}_\ell$ refines the residual; $\hat{\mathbf{z}} = \sum_{\ell=1}^{L} \mathbf{e}_\ell$. Unlike parallel subspace codes, each layer receives the residual left by its predecessors. Further training details are not covered here.

Li et al., *LSIG: Long Semantic IDs for Generative Recommendation*, WWW 2026.

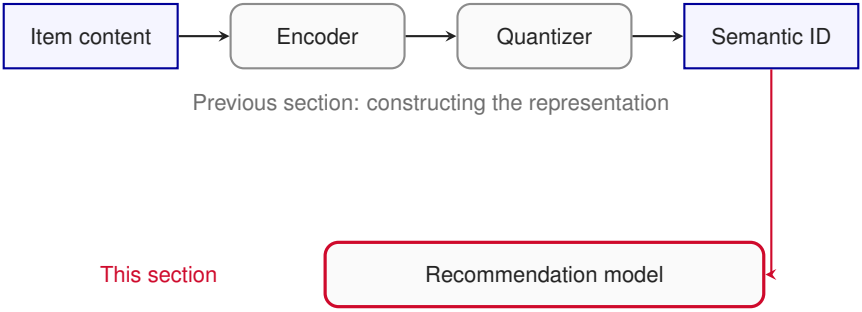
Summary: From Vector Quantization to Long Semantic IDs



Takeaway: OPQ and RQ provide two foundational paradigms for Semantic ID construction: parallel subspace decomposition and sequential residual refinement. Long IDs extend these structures. Arrows show design relationships, not a strict chronology.

Leveraging Semantic IDs in Recommender Systems

From SID Construction to Recommendation



The offline pipeline assigns item codes. Interaction data trains the recommendation model to use those codes.

A Semantic ID describes an item. A recommender learns what fits the user.

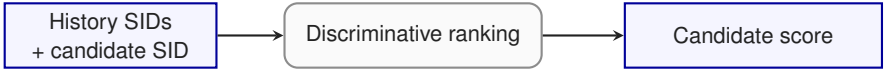
Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

Semantic IDs as Targets and Features



Target: predict the next item's SID, then resolve it to a catalog item.

Feature: consume a supplied candidate's SID to estimate its relevance.

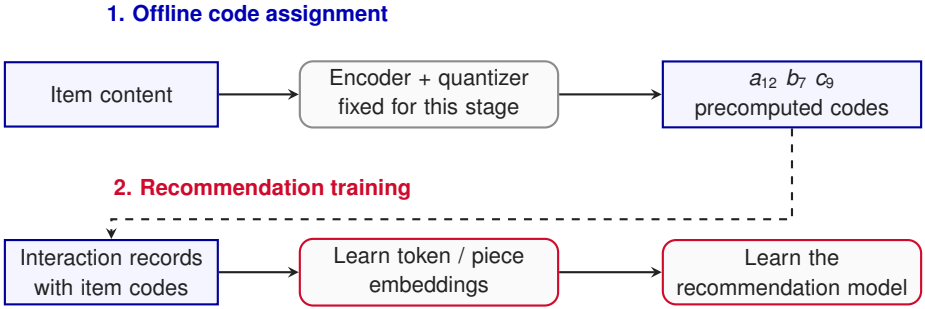


Both models learn from interactions; the SID plays a different role at the prediction interface.

The same representation family supports different prediction interfaces.

Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

What Is Fixed and What Is Learned?

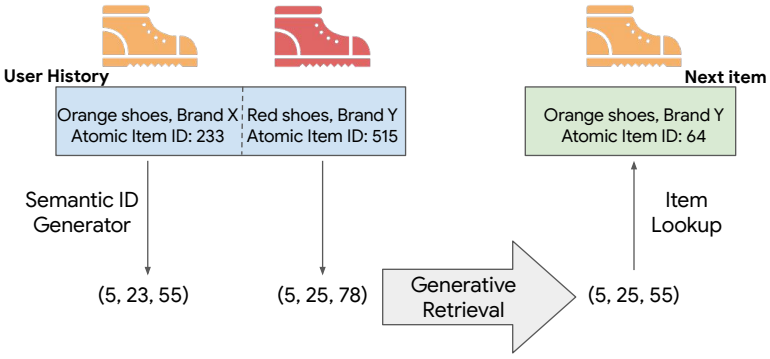


This section uses two-stage examples: fix the tokenizer, then train on interactions.

Quantization vectors approximate content; recommendation embeddings learn from interactions.

Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

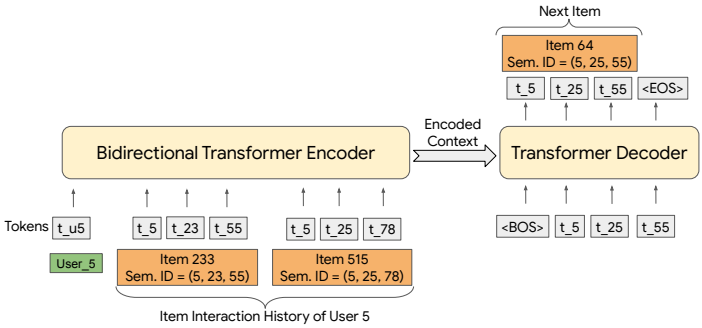
TIGER: Generating the Next Item's Identifier



Interaction history conditions the generation of a target SID.
A catalog lookup maps the generated identifier back to an item.

Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

TIGER: The Recommendation Model

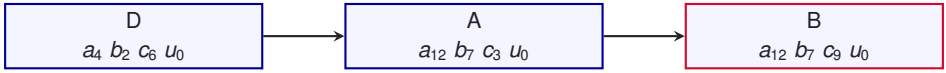


History tokens enter the encoder. The decoder predicts the next item's code sequence.

Interaction supervision connects the item representation to next-item prediction.

Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

An Interaction Sequence Becomes a Training Example



Encoder input $a_4\ b_2\ c_6\ u_0\quad a_{12}\ b_7\ c_3\ u_0$

Decoder target $a_{12}\ b_7\ c_9\ u_0\quad \mathrm{EOS}$

Supervision: B comes from the interaction log; the tokenizer supplies B's code.

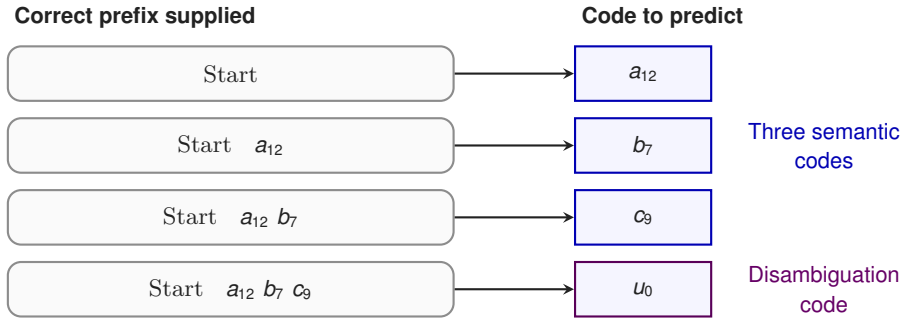
Each item contributes a fixed-length block. B is the label, outside the encoder history.

Preserve the interaction order when expanding items into token sequences.

Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

TIGER Training: Predicting the Next SID Token

History: **D, then A**. Observed next item: **B**, with code (a_{12}, b_7, c_9, u_0).

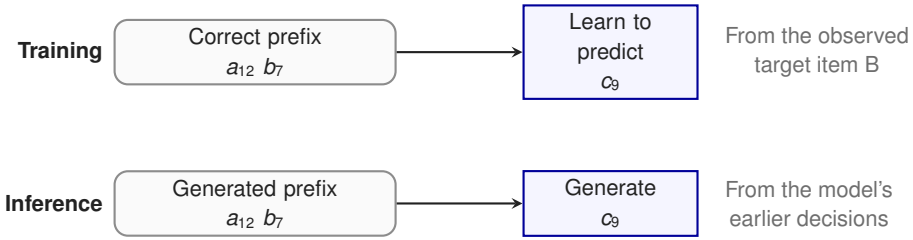


Teacher forcing supplies the correct prefix, without revealing the code being predicted.

Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

Autoregressive Recommendation at Inference

Autoregressive: generate one code at a time, using the codes already generated.



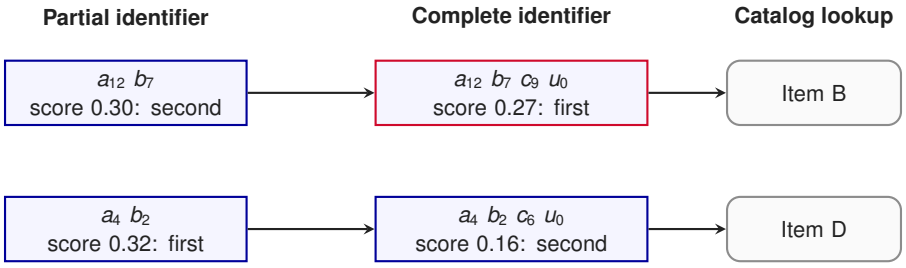
Start $\rightarrow$ a_{12} $\rightarrow$ $a_{12}\ b_7$ $\rightarrow$ $a_{12}\ b_7\ c_9$ $\rightarrow$ $a_{12}\ b_7\ c_9\ u_0$ $\rightarrow$ item B

At inference, an earlier mistake can change every later decision.

Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

Beam Search over Semantic IDs

Beam width = 2: temporarily keep two promising paths, instead of committing to one.



A path that trails early can win after more codes are generated.

Beam width counts retained paths; it is different from the number of recommendations requested.

Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

Resolving Generated Codes to Catalog Items

Situation	Example	Handling
Shared fragment	A and B share a_{12} b_7	Useful parameter sharing
Full semantic-code collision	B and E share a_{12} b_7 c_9	Append u_0 / u_1 for exact lookup
Invalid generated combination	a_{12} b_2 c_9 matches no item	Validate completed IDs and filter

A catalog map remains necessary even when retrieval generates identifiers.

Serving check: count valid, unique catalog hits before declaring that top-K items have been returned.

Shared semantic structure and unique catalog resolution are different requirements.

Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

Does the Semantic Structure Help?

Amazon Beauty: identifier ablation

Identifier	How are codes assigned?	Recall@5	nDCG@5
Random ID	Random code tuple; no content information	0.0296	0.0205
LSH SID	Locality-Sensitive Hashing: random projections of content vectors	0.0379	0.0259
RQ-VAE SID	Learned quantization of content representations	**0.0454**	**0.0321**

Same generative-retrieval framework; code capacities are comparable.

Evidence: TIGER, Table 2. Random ID here is a sequence of codes, not one atomic item token.

In this setting, content-organized codes outperform randomly assigned codes.

Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

Generative Retrieval: Benefits and Constraints

What SIDs enable

- Shared token parameters across related items
- A compositional target vocabulary
- Direct generation of item identifiers

What the system must handle

- Sequential decoding and beam-search cost
- Search pruning and invalid combinations
- Catalog resolution and item updates

Apply this to our catalog

A and B share a_{12}, b_7: their observations update common token parameters.

A newly encoded item can reuse those parameters, but useful full-ID generation still depends on training coverage, search and catalog resolution.

What to test: recommendation quality on new items, valid unique returns and latency at the chosen beam width.

Representing a new item and retrieving it effectively are separate capabilities.

Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

Discriminative Use: Retrieval and Ranking

Stage	What consumes the SID?	What is predicted?
Dense retrieval	Item-side representation for vector matching	Candidate similarity scores
Ranking	Candidate and history features in a ranker	Engagement or relevance scores

The consumption questions

- How should code-level embeddings be aggregated?
- How should SIDs interact with raw IDs and user history?

SID consumption includes aggregation, feature fusion and candidate matching.

Singh et al., *Better Generalization with Semantic IDs: A Case Study in Ranking for Recommendations*, RecSys 2024.

YouTube: SID Pieces as Trainable Features

Foundation: define pieces, look up embeddings, then aggregate

Adapter	Illustrative pieces of an eight-code SID
Unigram	$[a_{12}]$ $[b_7]$ $[c_9]$ $[d_4]$ $[e_2]$ $[f_6]$ $[g_1]$ $[h_8]$
Bigram	$[a_{12}\ b_7]$ $[c_9\ d_4]$ $[e_2\ f_6]$ $[g_1\ h_8]$
Learned pieces	$[a_{12}\ b_7\ c_9]$ $[d_4]$ $[e_2\ f_6]$ $[g_1\ h_8]$

The piece vocabulary controls sharing and memorization.

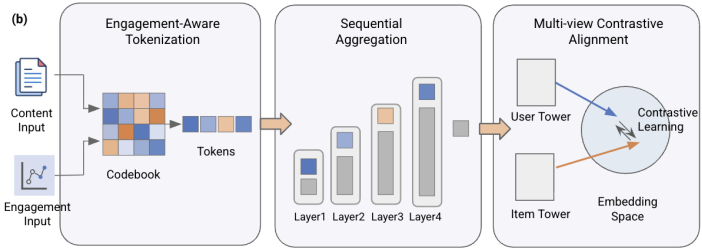
The next question is how to combine the resulting features.

Design trade-off: shorter pieces share more parameters; longer pieces distinguish finer combinations.

Piece construction defines the features available to the downstream model.

Singh et al., *Better Generalization with Semantic IDs: A Case Study in Ranking for Recommendations*, RecSys 2024.

Pin-SCALE: Integrating SIDs into Dense Retrieval



Three integration decisions

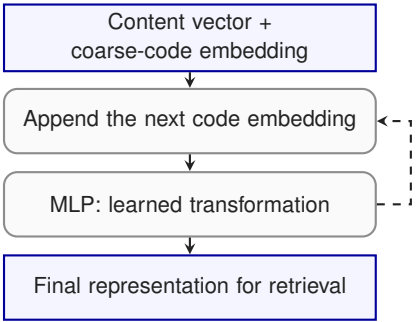
- **Tokenize:** choose representations that capture content and engagement.
- **Aggregate:** progressively combine SID levels into a retrieval representation.
- **Align:** train with cross-tower and co-occurrence contrastive objectives.

Tokenization, aggregation and task alignment jointly determine SID utility.

Qu et al., *Pin-SCALE: Semantic Cascading and Alignment Learning for Engagement-Aware IDs in Cold-Start Recommendations*, SIGIR 2026.

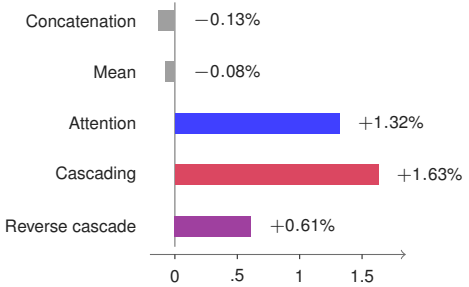
Pin-SCALE: Cascading Pooling, Step by Step

Build the representation in order



Repeat append + MLP from coarse to fine.

Closeup online pooling study

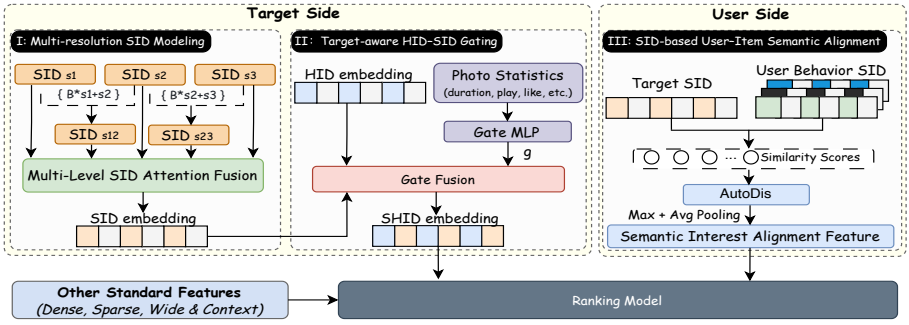


Fresh Repin = saves. Relative lift versus production without SID inputs (Table 1).

Both the aggregation operator and its order matter in this reported setting.

Qu et al., *Pin-SCALE: Semantic Cascading and Alignment Learning for Engagement-Aware IDs in Cold-Start Recommendations*, SIGIR 2026.

SID-Coord: Coordinating SID Consumption



HID = Hashed Item ID. Code resolutions, HID–SID fusion and history alignment feed the ranker.

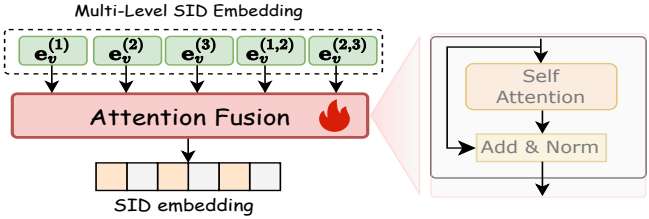
SID-Coord makes feature consumption explicit at several points in the ranking model.

Li et al., *SID-Coord: Coordinating Semantic IDs for ID-based Ranking in Short-Video Search*, SIGIR 2026.

SID-Coord: Multi-Resolution Attention Fusion

Three base codes become five representation views

$s^{(1)}$, $s^{(2)}$, $s^{(3)}$, $s^{(1,2)}$, $s^{(2,3)}$



Adjacent-level compositions add finer views. Attention learns how to fuse their embeddings.

A model can learn the useful semantic resolution instead of fixing one pooling granularity.

Li et al., *SID-Coord: Coordinating Semantic IDs for ID-based Ranking in Short-Video Search*, SIGIR 2026.

SID-Coord: Gating Memorization and Generalization

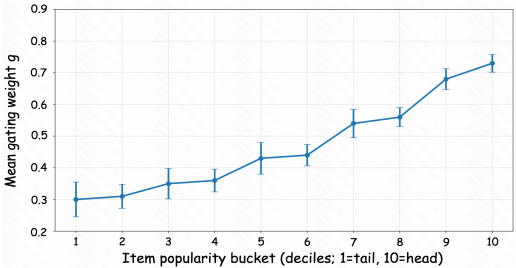
HID = Hashed Item ID

Historical item statistics drive a small gate network.

g: weight on the **HID branch**.

$1 - g$: weight on the **SID branch**.

More observed behavior can support stronger item-specific memorization.

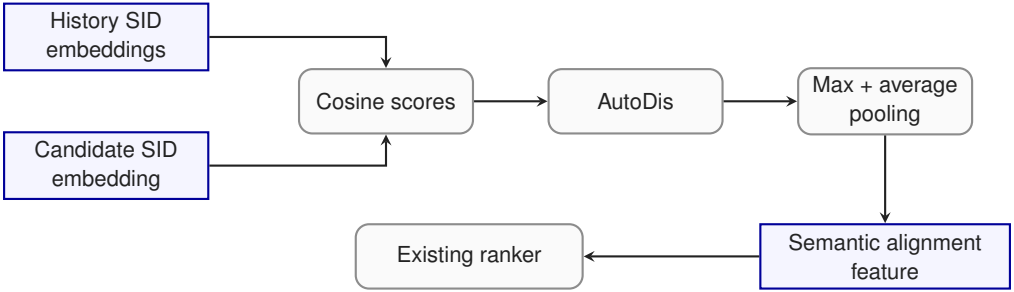


Mixed result: $g \times$ HID embedding $+ (1 - g) \times$ SID embedding, $0 \leq g \leq 1$

The gate coordinates ID memorization with semantic sharing for each target item.

Li et al., *SID-Coord: Coordinating Semantic IDs for ID-based Ranking in Short-Video Search*, SIGIR 2026.

SID-Coord: Candidate–History Alignment Features



A different candidate induces different similarity scores over the same history.

SIDs can provide explicit matching features in addition to item embeddings.

Li et al., *SID-Coord: Coordinating Semantic IDs for ID-based Ranking in Short-Video Search*, SIGIR 2026.

SID Consumption: Mechanisms and Evidence

What does the model consume?

Case	Consumption focus
YouTube	Piece embeddings
Pin-SCALE	Hierarchy-preserving pooling for retrieval
SID-Coord	Resolution fusion, ID gating and matching features

SID-Coord ablation: overall AUC

Configuration	AUC
Full model	**0.7708**
Without multi-resolution	0.7691
Without HID–SID gate	0.7698
Without alignment	0.7696

Read within each system: SID-Coord isolates consumption modules; Pin-SCALE compares aggregation choices. Their metrics and workloads differ.

Ablate consumption modules as well as tokenizers when evaluating SID integration.

Li et al., *SID-Coord: Coordinating Semantic IDs for ID-based Ranking in Short-Video Search*, SIGIR 2026.

What Explains an Improvement?

Code structure: semantic codes beat random codes in the Beauty ablation.

Rajput et al. (2023), TIGER, Table 2

Consumption: removing fusion, gating or alignment lowers overall AUC in SID-Coord.

Li et al. (2026), SID-Coord, Table 2

Content alone: directly substituting dense content does not automatically improve AUC.

Singh et al. (2024), YouTube ranking, Figure 2: without history

Suggested experiment: match information, model and resource budgets before attributing gains to the ID.

These are results in specific settings; controlled comparisons establish what caused the gain.

Evaluation beyond the Overall Average

SID diagnostics

- Code usage and collision rates
- Semantic neighborhoods
- Stability of item assignments

Check whether codes are well used and group related items.

Recommendation outcomes

- Recall / nDCG for retrieval
- AUC / LogLoss for ranking
- Overall, head / tail, new-item slices

Check gains for sparse and new items under the same protocol.

Make the new-item protocol explicit

Tokenizer: content seen during fitting? **Recommender:** training interactions seen?

An item can be new to the recommender while its content was already visible to the tokenizer.

A good-looking code space does not by itself establish better recommendations.

Singh et al., *Better Generalization with Semantic IDs: A Case Study in Ranking for Recommendations*, RecSys 2024.

Recommendation Quality and System Cost

Measure	Generative retrieval	Discriminative ranking
Quality	Recall / nDCG; valid returned items	AUC / LogLoss; quality by item slice
Inference effort	ID length and beam width	Piece lookups and history processing
Memory / storage	Token tables + model + catalog map	Piece tables + model + SID catalog
Serving behavior	Latency and throughput at target K	Latency and throughput at candidate load

Report the operating point: hardware, batch size, history length and candidate count.

Practical comparison: match recommendation quality, then compare end-to-end latency and memory at that operating point.

Compare quality and cost together under an explicit workload.

Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

Generating IDs vs. Scoring Candidates

	Generate an identifier	Score a candidate
Inputs	History SID tokens	History + current item + candidate
Output	A generated ID, then item lookup	A score used to order candidates
Shared parameters	Recommendation token parameters	Piece / level embeddings and fusion modules
Key design choice	Tokenizer and decoding strategy	Aggregation, fusion and feature integration
Main constraint	Search quality and resolution cost	Sharing, memorization and serving cost

They can coexist: retrieve a candidate set with generated IDs, then rank it using SID-derived features.

The useful interface depends on the recommendation task and existing pipeline.

Rajput et al., *Recommender Systems with Generative Retrieval*, NeurIPS 2023.

Overview & Agenda

Part I: Foundations 85m

- Introduction to recommender systems, raw IDs [25m]
- Semantic IDs: introduction and overview [25m]
- Leveraging semantic IDs in recommender systems [35m]

Part II: Applications 95m

- Hands-on: building and using semantic IDs [35m]
- Multimodality and collaborative signals [30m]
- Cold-start mitigation and cross-domain transfer [30m]

Part III: Perspectives 60m

- Hands-on: cross-domain usage of Semantic IDs [30m]
- Future directions and open challenges [15m]
- Q&A and open discussion [15m]

14:00–18:30 · 4 hours of content + 30-minute tea break (16:00–16:30)

Hands-On 1: Building and Using Semantic IDs

Hands-on notebooks

Session 1: Building and using semantic IDs

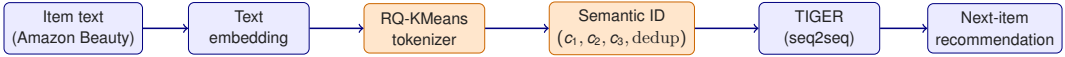


`https://semantic-id-tutorial.github.io/hands-on.html`

Open the notebooks in Google Colab

Sign in with your Google account.

From Text to Semantic IDs to Generative Recommendation



Part 1 – Build Semantic IDs

- Encode item text into embeddings
- Train an **RQ-KMeans** tokenizer
- Turn each item into a short tuple of discrete codes
- Handle collisions with a *dedup* level
- Notebook in Colab

Part 2 – TIGER vs. SASRec

- Load pretrained models (no training in the room)
- Evaluate: Recall@10, NDCG@10, MRR
- Compare model size and #parameters
- **Play with TIGER**: constrained beam search, SIDs $\rightarrow$ items
- Notebook in Colab

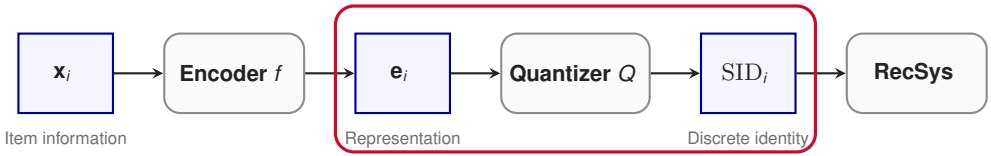
Setup: notebooks run on **Google Colab** with `Cornac`; no local install needed. **Take-home:** tune TIGER via `train_and_save_models.py`, aim for Recall@10 ≈ 0.065.

Part 2

Applications

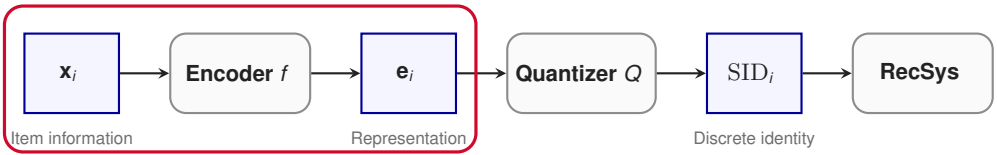
Multimodality and Collaborative Signals

Recap: Quantization



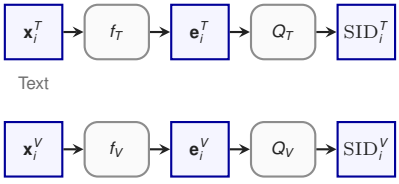
In the previous sessions, we introduced how **quantization** maps continuous item representations to discrete **Semantic IDs**.

Inputs to Semantic ID Construction

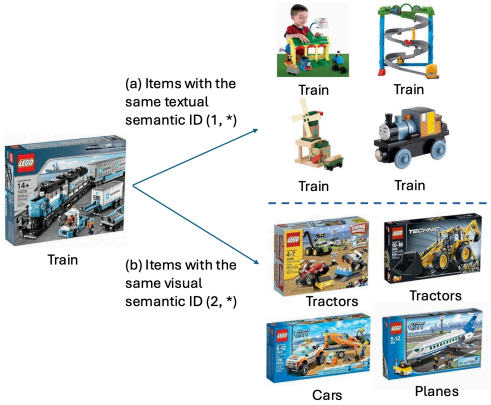


In this session, we focus on **different input modalities**, how to **combine them**, and how to incorporate **behavioral signals** into Semantic ID construction.

Different Modalities Reveal Different Semantics



Text and images can reveal **different similarities** between the same items.

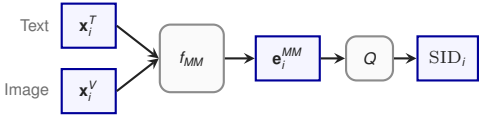


Same item, different modality-specific neighbors.

Zhu et al., *Beyond Unimodal Boundaries: Generative Recommendation with Multimodal Semantics*, 2025, Fig. 4.

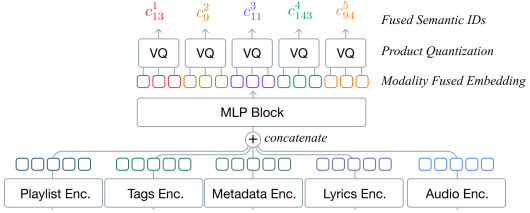
Early Fusion: Fuse, Then Quantize

Early Fusion



- **Fuse before quantization** into one SID.
- Learns **cross-modal interactions** in a joint representation.
- Risk: dominant modalities can obscure specific details.

Example: FusID

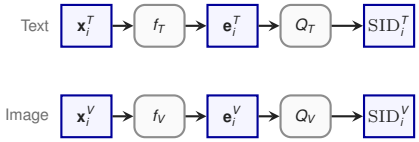


Separate encoders → concatenate + MLP → quantize.

FusID, 2026, Fig. 1 (excerpt); arXiv:2601.08764.

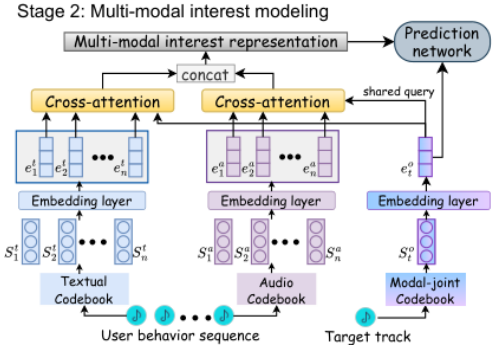
Late Fusion: Quantize, Then Fuse

Late Fusion



- **Quantize each modality separately.**
- Preserve distinct semantics; **fuse SIDs downstream**.
- Challenge: learn relationships across separate SID spaces.

Example: PSRQ

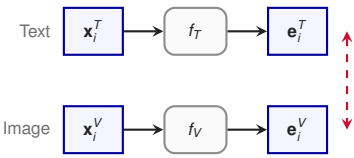


Hybrid tokenizer; shown here: downstream fusion of SID streams.

Wang et al., *PSRQ*, CIKM 2025, Fig. 3, Stage 2 (excerpt).

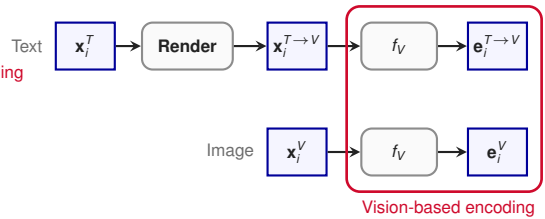
Fusion Also Depends on the Encoders

Separate Encoders



- Different training objectives can yield **different embedding spaces**.
- Fusion must learn how those spaces relate.

Text-as-Vision

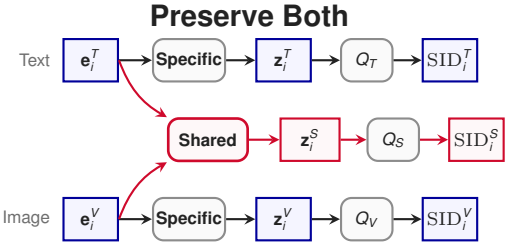


- Render text as an image; use a **vision-based OCR encoder**.
- Text and images follow a more similar encoding pathway.

Encoder choice shapes how easily modalities can be combined.

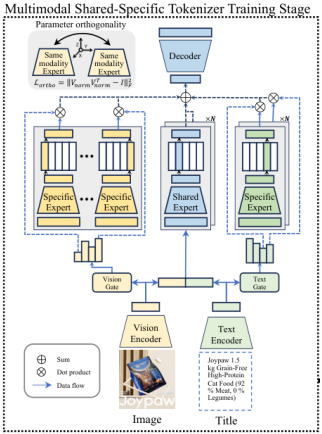
Qiao et al., *When Text-as-Vision Meets Semantic IDs*, 2026.

Shared and Modality-Specific Semantics



- **Shared streams** capture common semantics.
- **Specific streams** retain each modality's details.
- Combine both to capture **commonality and complementarity**.

Example: MMQ

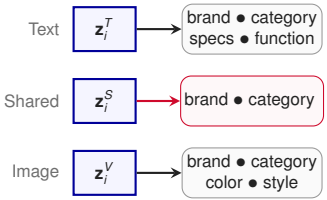


Gates combine shared and modality-specific experts.

Xu et al., *MMQ*, 2025, Fig. 2, tokenizer training stage (excerpt).

More Modalities Should Add New Information

Repeated Semantics



- Repeating **brand** and **category** across streams uses capacity without adding new information.

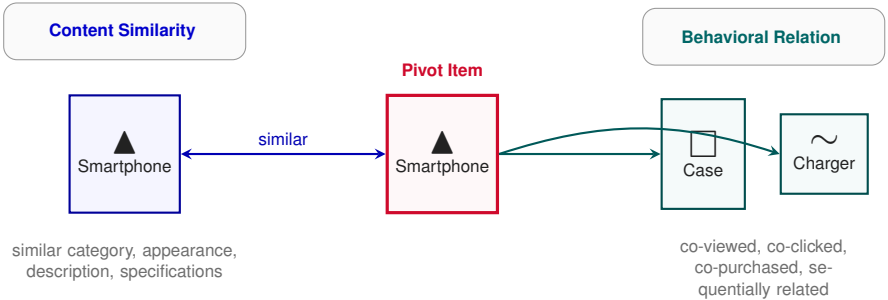
Encourage Distinct Signals

- **Separate shared and specific information.** Give common semantics and modality-specific details distinct roles.
- **Different expert transformations (MMQ).** Orthogonality regularization encourages experts to differ.
- **Less correlated sub-embeddings (FusID).** Covariance regularization discourages duplicated signals.

More modalities $\neq$ more information — each added stream should contribute **complementary information**.

Xu et al., *MMQ*, 2025; Kim et al., *FusID*, 2026.

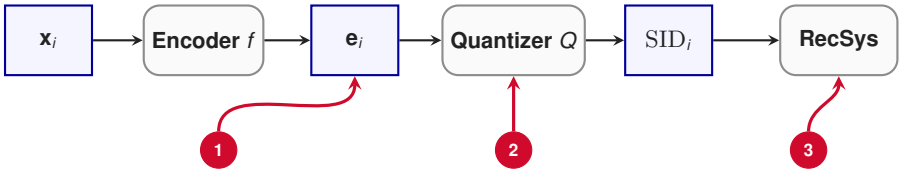
Recommendation Needs Behavior



Content describes the item ≠ **Behavior reveals** relationships between items

Content similarity alone does not determine **recommendation relevance**.

Where Can Behavior Shape Semantic IDs?



Before Quantization

Align item embeddings with behavior.

During Quantization

Use behavior to learn codes.

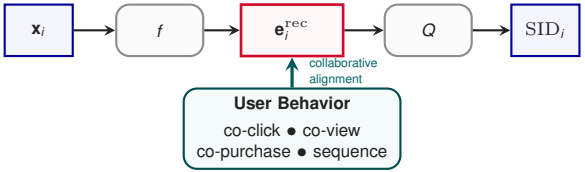
End-to-End

Let recommendation loss update the tokenizer.

Behavior can shape **embeddings**, **codes**, or **joint training**.

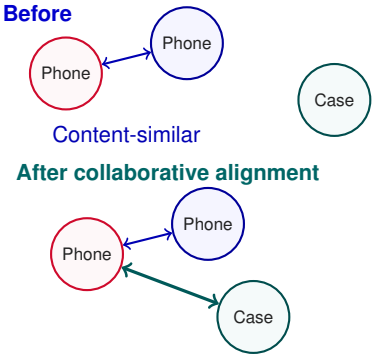
Before Quantization — Make the Embedding Recommendation-Aware

Inject Behavior into the Representation



- **Behavioral item pairs** supervise the content representation.
- Alignment training captures **recommendation-relevant relationships**.
- **Quantize the aligned embeddings** into Semantic IDs.

Example: QARM

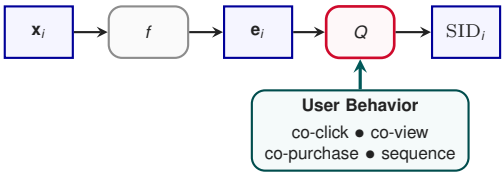


Behavior aligns embeddings before quantization (illustrative).

Luo et al., *QARM: Quantitative Alignment Multi-Modal Recommendation at Kuaishou*, 2024.

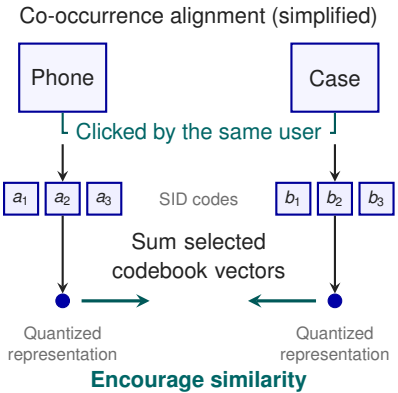
During Quantization — Make the Discrete Space Behavior-Aware

Inject Behavior into Quantization



- **Behavior supervises codebook learning.**
- Balance **content reconstruction** and **behavioral alignment**.
- Learn **recommendation-aware discrete representations**.

Example: DAS

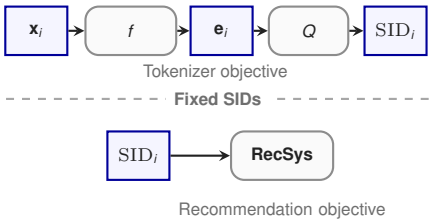


DAS uses co-occurrence to align quantized item representations during codebook learning.

Ye et al., *DAS: Dual-Aligned Semantic IDs Empowered Industrial Recommender System*, CIKM 2025.

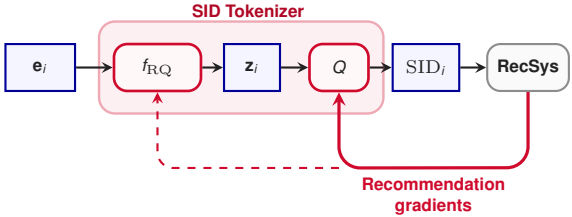
End-to-End — Let Recommendation Shape the SID

Separate Training



- **Train the tokenizer first**, fix the SID assignments, then train the recommender.
- **Objective mismatch:** recommendation errors cannot improve the tokenizer.

Joint Training — Example: DIGER

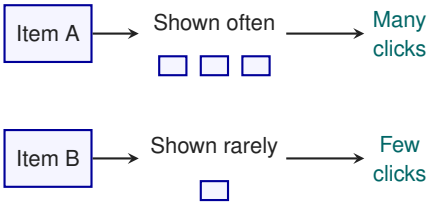


DIGER approximates gradients through discrete SID assignments, jointly training the tokenizer's encoder and codebooks with the recommender.

Fu et al., *Differentiable Semantic ID for Generative Recommendation (DIGER)*, 2026.

Observed Behavior Also Carries Bias

Unequal Exposure



Different click counts alone do not establish which item users prefer.

Align to Reliable Signals

- Clicks reflect **exposure as well as interest**.
- Alignment can carry **existing biases into SIDs**.
- Account for bias when constructing **behavioral supervision**.

Example: DAS separates interest-related signals from popularity and conformity before aligning representations.

The **behavioral signal** we align to also matters.

Ye et al., *DAS: Dual-Aligned Semantic IDs Empowered Industrial Recommender System*, CIKM 2025, Sec. 2.2.

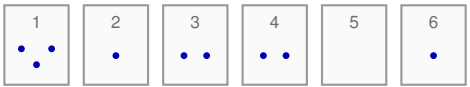
Joint Training Can Leave Most Codes Unused

Code Usage in One Codebook

Few active codes



Broader code use



Dots = assigned items.
Schematic usage at one codebook level.

Keep the Codes Useful

- Early assignments can become **overconfident**.
- Severe underuse wastes **representation capacity**.
- Monitor **code utilization and recommendation quality** together.

Example: DIGER encourages early exploration, then gradually reduces assignment randomness.

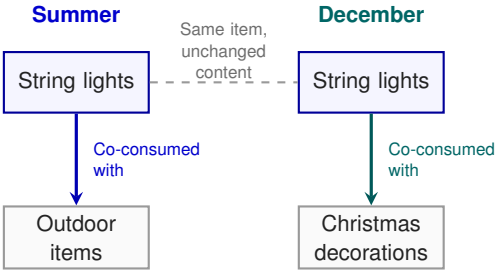
Perfectly uniform code usage is not the goal.

Gradient flow alone does not ensure useful codes.

Fu et al., *Differentiable Semantic ID for Generative Recommendation (DIGER)*, 2026.

Same Content, Changing Behavior

Changing Co-occurrence Patterns



The item stays the same;
its behavioral context can change.

Adapt Without Needless Changes

- Behavior-aware codes reflect **past interaction patterns**.
- Updating the tokenizer can **reassign existing IDs**.
- Adapt selectively and **coordinate tokenizer and recommender updates**.

Example: DACT adapts drifting items while limiting unnecessary code changes.

Fixed IDs can still support an updated recommender.

Balance **adaptation to new behavior** with **identifier stability**.

Feng et al., *Drift-Aware Continual Tokenization for Generative Recommendation (DACT)*, 2026 preprint.

Takeaways: Multimodal Content and Behavior

MULTIMODAL CONTENT

- **Complementary information:** different modalities should contribute distinct, useful signals.
- **Fusion design:** combine modalities before quantization, downstream, or through shared and specific streams.

COLLABORATIVE SIGNALS

- **Beyond content similarity:** behavior reveals relationships relevant to recommendation.
- **Where to intervene:** align embeddings, shape codebook learning, or jointly train tokenizer and recommender.

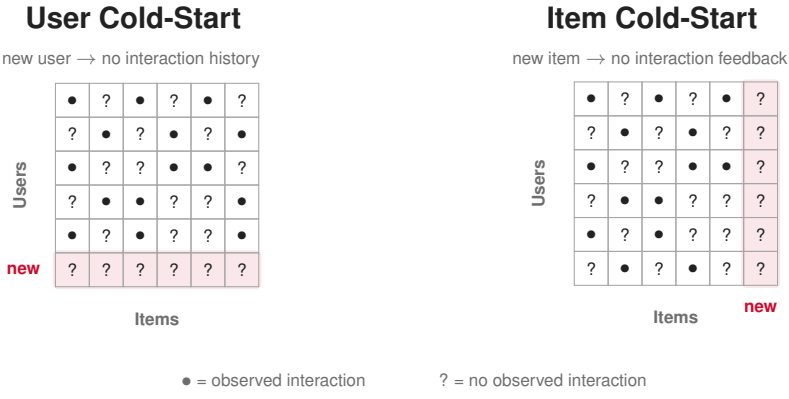
DESIGN CONSIDERATIONS

- **Preserve semantics:** balance behavioral alignment with meaningful content structure.
- **Check reliability:** consider biased supervision, code underuse, and changing behavior.

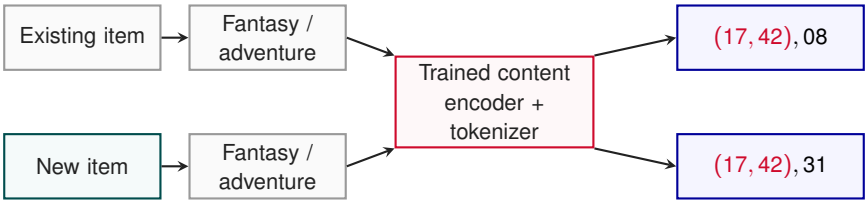
Cold-Start Mitigation and Cross-Domain Transfer

Cold-Start: When Behavioral Signals Are Missing

- **Behavior complements content:** interactions can reveal relationships that content alone may miss.
- **Cold-start:** behavioral evidence is **missing or scarce** — most commonly for **new users** or **new items**.



Cold-Start: Reuse Learned Semantic Structure

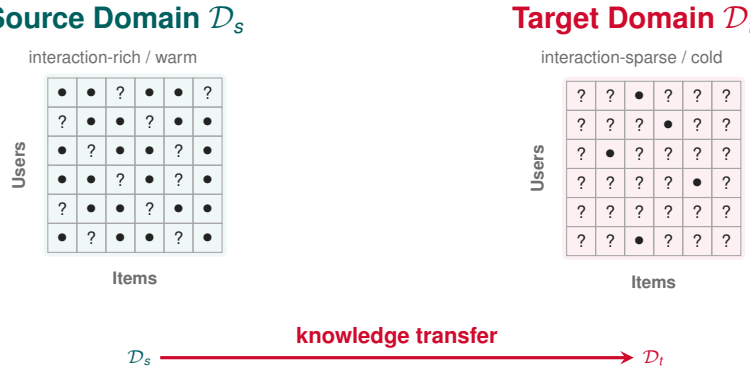


Illustrative codes: shared semantic structure, item-specific detail.

- **Before interactions accumulate:** item content can already provide a Semantic ID.
- **Reuse learned structure:** related items can share codes learned from existing items.
- **Representation is one part of the task:** user preferences and downstream recommendation still need modelling.

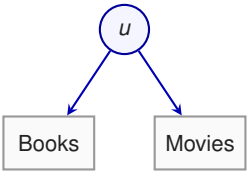
Cross-Domain Recommendation: Learn Here, Help There

- Cold-start can also occur at the **domain** / **category level**: one domain may be interaction-rich while another remains sparse or cold.
- **Cross-Domain Recommendation (CDR)** transfers knowledge from a **source domain** to improve recommendation in a **target domain**.



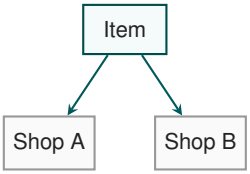
What Connects the Domains?

Shared Users



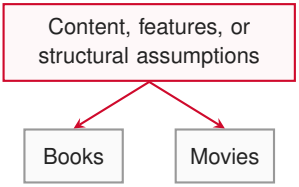
The same users provide cross-domain anchors.

Shared Items



Common items connect different user populations.

Neither Users nor Items



One option: related content connects domains without shared identities.

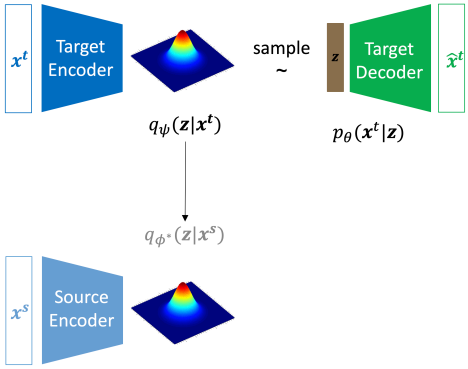
- **Overlap determines available anchors:** it matters for method choice and evaluation.
- **Our examples:** SA-VAE uses shared users; GMC uses item text without explicit cross-domain user identity links.

Zang et al., *A Survey on Cross-domain Recommendation*, ACM TOIS 2023; Salah et al., 2021; Jin et al., 2025.

Traditional CDR: Align User Preference Spaces

Example: SA-VAE — Shared Users

- **Same users, different catalogues:** Book and Movie interactions describe the same users.
- **Source preferences:** learn a user-preference distribution from Book interactions.
- **Soft alignment:** guide the target Movie preference distribution toward the source.
- **Target prediction:** decode the aligned preferences over Movie items.



Book and Movie item IDs remain **separate**; knowledge is transferred through the **aligned user-preference space**.

Salah et al., *Towards Source-Aligned Variational Models for Cross-Domain Recommendation*, RecSys 2021.

Limitations of Explicit Cross-Domain Alignment

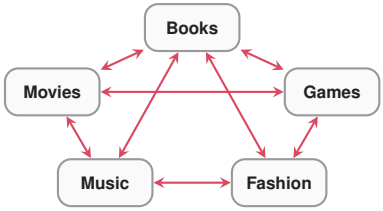
- **Atomic IDs are domain-specific:** cross-domain similarity is not encoded in the identifiers themselves.
- **Alignment-based approaches** construct an explicit model-level bridge between domains.
- Maintaining **domain-specific bridges** can become harder as the number of domains grows.

Two Domains



One explicit bridge is manageable.

Many Domains

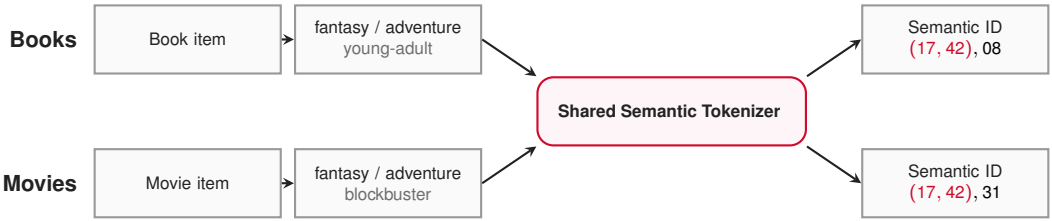


More domains → more transfer relationships and coordination.

Can we encode the shared structure directly in the **item representation** instead?

A Shared Semantic Vocabulary Across Domains

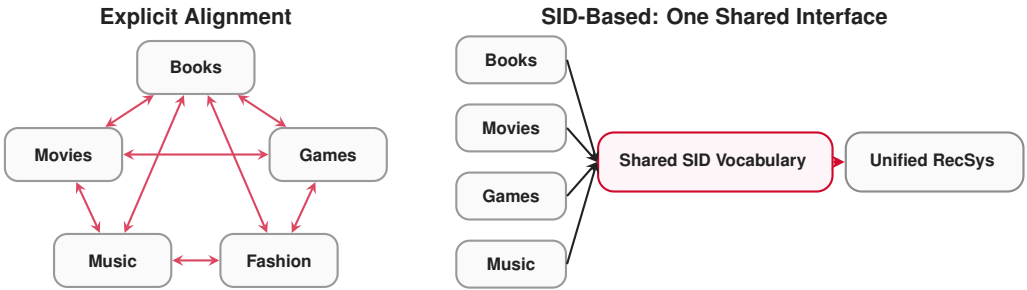
- Map items from all domains into a **shared semantic vocabulary** as a common interface for transfer.
- Related items can reuse **common semantic tokens**, while preserving **item-specific detail**.



Shared tokens such as $(17, 42)$ can capture **transferable semantics**; different tokens preserve **item-specific information**.

From Pairwise Bridges to a Shared Interface

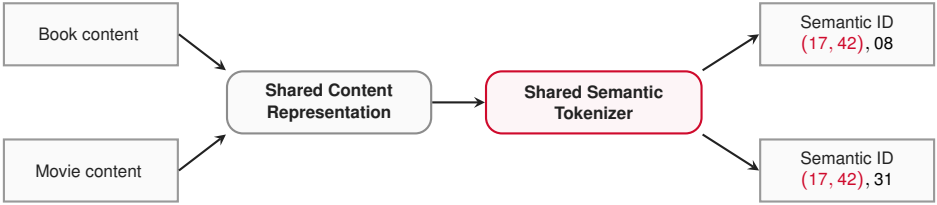
- **Alignment-based CDR:** transfer through explicit domain-level bridges.
- **Shared SIDs:** encode cross-domain structure in a common item representation used by a shared recommender.



A shared **semantic vocabulary and backbone** can reduce reliance on explicit pairwise bridges; domain adaptation may still be needed.

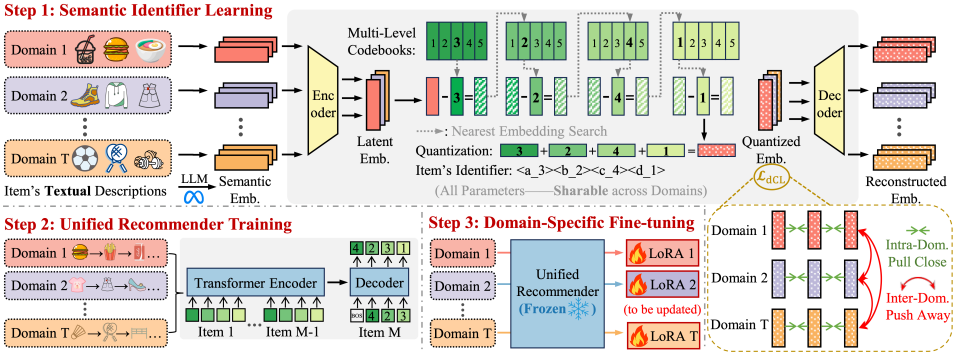
Shared Vocabulary Requires Shared Semantics

- **Content distributions can differ:** vocabulary, attributes, and description quality vary across domains.
- **Useful sharing requires comparable semantics:** the encoder and tokenizer must preserve meaningful relationships across these differences.



- **Same vocabulary $\neq$ useful transfer:** poor representations or tokenization can cause **false sharing** and negative transfer.
- The design illustrated next combines **cross-domain semantics, shared tokenization, and unified recommendation**.

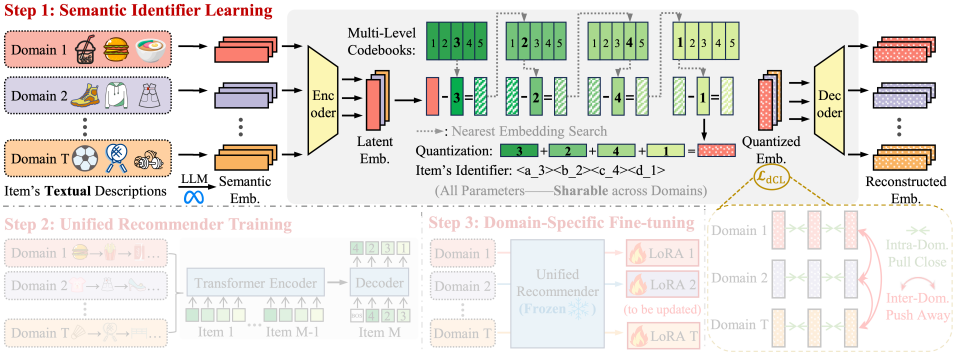
GMC: A Concrete Cross-Domain SID Architecture



- **Multi-target CDR with GMC:** shared text representation → shared SIDs → unified recommender → lightweight domain adaptation.

Jin et al., *Generative Multi-Target Cross-Domain Recommendation*, 2025.

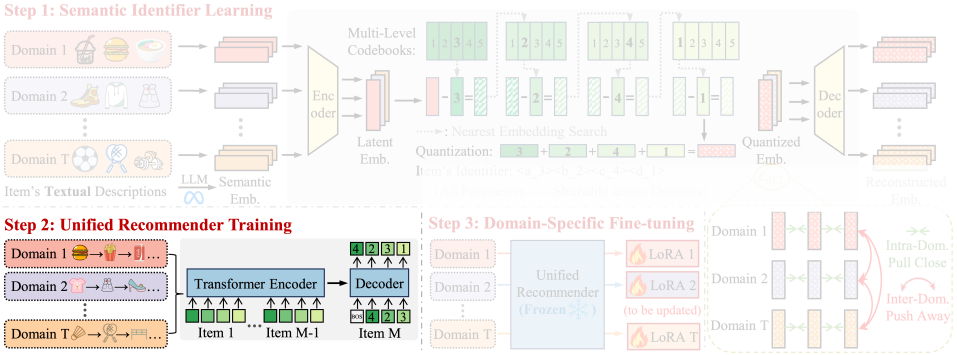
GMC: Step 1 — Semantic Identifier Learning



- **Shared representation:** one LLM embeds item text from every domain into a common semantic space.
- **Shared SID learning:** one RQ-VAE uses shared codebooks and a domain-aware contrastive objective.

Jin et al., *Generative Multi-Target Cross-Domain Recommendation*, 2025.

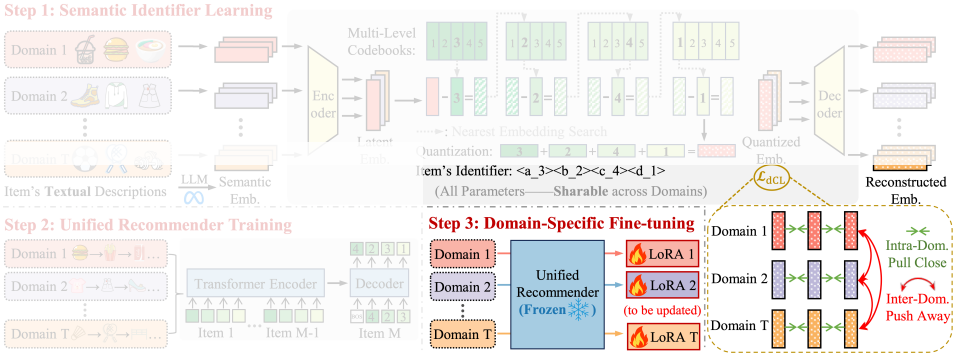
GMC: Step 2 — Unified Recommender Training



- **Unified training:** SID interaction sequences from all domains jointly train one shared recommender.
- **Generative prediction:** the model predicts the next item directly as its multi-level Semantic ID.

Jin et al., *Generative Multi-Target Cross-Domain Recommendation*, 2025.

GMC: Step 3 — Domain-Specific Fine-Tuning



- **Shared backbone stays fixed:** the unified recommender is frozen after multi-domain training.
- **Lightweight specialization:** each domain learns a LoRA adapter from its own interaction data.

Jin et al., *Generative Multi-Target Cross-Domain Recommendation*, 2025.

Takeaways: Semantic IDs for Cold-Start & Cross-Domain Recommendation

SHIFT

- **Alignment-based CDR:** transfer mainly through **model-level alignment**.
- **SID-based transfer:** cross-domain structure enters already through the **item identity**.

CONSEQUENCES

- **Item cold-start:** content can provide an informative identity before interactions accumulate.
- **Cross-domain:** shared SIDs expose reusable semantic structure across domains.
- **Practical design:** share vocabulary and backbone; specialize only where needed.

CAVEAT

- Shared codes help only when the underlying representations capture **genuinely comparable semantics**.

Scope: What We Did Not Cover

- **Cold-start regimes:** user, item, session/context, and domain-level cold-start.
- **Overlap assumptions:** shared users/items, partial overlap, or neither determine available transfer mechanisms and evaluation settings.
- **Other transfer mechanisms:** shared features/users, domain adaptation, meta-learning, graph and behavioral transfer.
- **Evaluation challenges:** negative transfer, domain imbalance, sparsity, and generalization to unseen domains.

- **Our focus:** how content-derived **Semantic IDs** can provide a **shared representation and transfer interface** across domains.

Overview & Agenda

Part I: Foundations 85m

- Introduction to recommender systems, raw IDs [25m]
- Semantic IDs: introduction and overview [25m]
- Leveraging semantic IDs in recommender systems [35m]

Part II: Applications 95m

- Hands-on: building and using semantic IDs [35m]
- Multimodality and collaborative signals [30m]
- Cold-start mitigation and cross-domain transfer [30m]

Part III: Perspectives 60m

- Hands-on: cross-domain usage of Semantic IDs [30m]
- Future directions and open challenges [15m]
- Q&A and open discussion [15m]

14:00–18:30 · 4 hours of content + 30-minute tea break (16:00–16:30)

Part 3

Perspectives

Hands-On 2: Cross-Domain Usage of Semantic IDs

Session 2: Cross-domain transfer

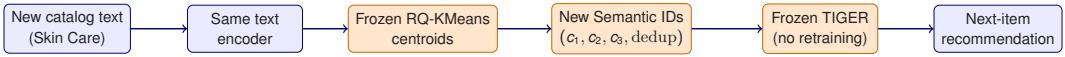


`https://semantic-id-tutorial.github.io/hands-on.html`

Open the notebooks in Google Colab

Sign in with your Google account.

Cross-Domain Transfer with Semantic IDs



In the room – Transfer without training

- Load the Session 1 TIGER (**source**) and evaluate it in-domain
- Carve out a **target domain**: Beauty → Skin Care
- Embed target items and quantize them with the source's **frozen** centroids
- Plug the new SIDs into the frozen transformer and evaluate
- Notebook in Colab

Take-home exercises

- **Exercise 1:** handle higher collision rates when the target needs more *dedup* slots than the source reserved
- **Exercise 2:** true cold start: retrain the source *without* Skin Care, then transfer and compare

Setup: notebooks run on **Google Colab** with `Cornac`; no local install needed.

Key idea: SIDs are content-derived, so a new catalog only needs *re-tokenizing*.

Future Directions and Open Challenges

What Remains Open?

From working mechanisms to defensible design choices



- What information should IDs share, and what must stay distinct?
- What evidence survives changes in the catalog and model?
- Who benefits, and what should the next exposure accomplish?

The open question is when an ID design is useful, for whom, and for how long.

What Should an ID Share—and Preserve?

A new product resembles a popular one. Should they share evidence?

Similar descriptions can coexist with different prices, availability and user responses.

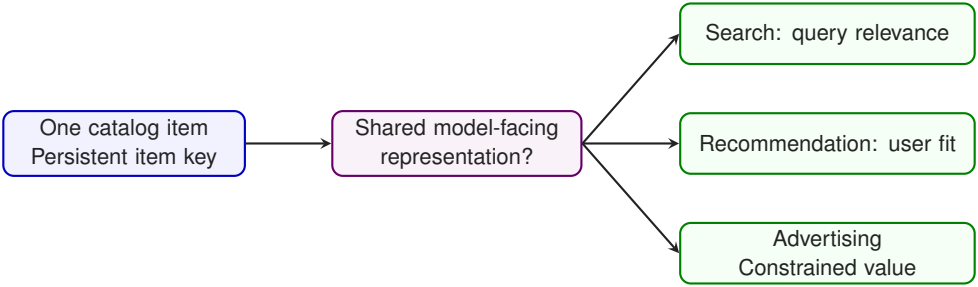
Design	What it preserves / shares	Question to investigate
Raw ID	Item-specific parameters	Can available interactions support each item?
Semantic ID	Reusable structured components	Which similarities justify transferring evidence?
Hybrid ID	Atomic identity plus shared components	When should each component influence prediction?

Design choice: encode a signal in the ID, or supply it as a downstream feature?

Semantic similarity does not guarantee interchangeable user responses.

Chen et al., *What Makes a Good Semantic ID for Generative Recommendation? A Reproducibility Study*, 2026.

Can One Identity Space Serve Multiple Tasks?



Open questions: which components should be shared? Where is task-specific information needed?

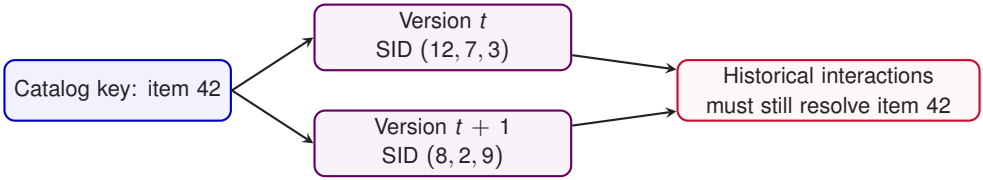
Test positive transfer and negative transfer, rather than assuming a common vocabulary is enough.

Sharing an identity space does not establish that task objectives are aligned.

Unified Search and Recommendation Workshop (USRW), RecSys 2026.

What Must Survive an ID Update?

The item persists; test strategies for changing its code.



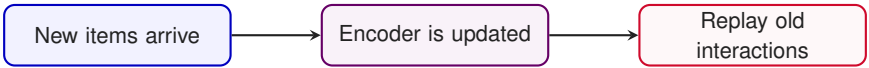
- **Version the release:** encoder, codebook, mappings and compatible recommender.
- **Preserve resolvability:** log catalog keys; replay through versioned mappings.
- **Test the migration:** shadow-test versions; define retraining, rollout and rollback.

Stable catalog identity and adaptable model-facing IDs can coexist.

Berners-Lee, *Cool URIs Don't Change*, W3C, 1998.

What Evidence Would Justify an ID Choice?

Compare a lifecycle, not only a frozen benchmark.



- **Control:** input information, backbone, training data and serving budget.
- **Measure:** quality by slice, correct item resolution, latency and update cost.
- **Report:** failures, negative transfer and when the baseline remains preferable.

A proposed shared test case, not a benchmark result.

An ID choice needs attributable benefits and an explicit maintenance cost.

Chen et al., *What Makes a Good Semantic ID for Generative Recommendation? A Reproducibility Study*, 2026.

Beyond Accuracy: Who Benefits from Semantic IDs?

Beyond-accuracy objectives are established; SID-specific evidence remains incomplete.

Perspective	What to inspect	Open question
User fairness	Utility across users / groups	Who gains, and whose quality deteriorates?
Provider fairness	Exposure and opportunities	What allocation standard is justified?
Coverage / diversity	Breadth of recommended items	Which items and interests remain unreachable?

SID research already includes diversity and debiasing studies.

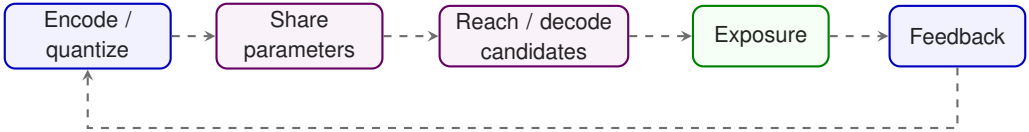
Codebook utilization, catalog coverage and fairness measure different things.

A higher average score does not show how benefits and opportunities are distributed.

McNee et al., *Being Accurate Is Not Enough: How Accuracy Metrics Have Hurt Recommender Systems*, CHI 2006.

Does Semantic Sharing Also Amplify Bias?

A mechanism to investigate—not an established causal result

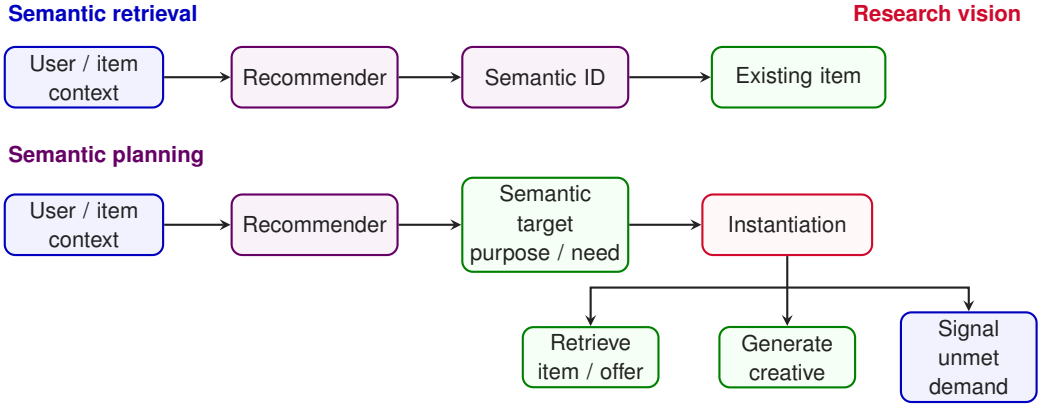


- Could frequent prefixes and pruning disadvantage niche content and its users?
- Does a token-balancing change alter item exposure and user-group utility?
- Hold the backbone fixed; compare stages and declare the fairness criterion.

Balanced tokens do not guarantee fair outcomes for users or providers.

Li et al., *The First International Workshop on Item IDentifiers Design, Evaluation, and Adaptation (IDEA)*, workshop proposal.

From Item Retrieval to Exposure Planning



Semantic planning is a hypothesis about an additional decision layer.

Jin et al., *From Raw IDs to Semantic Planning: How Recommender Systems Utilize Information at Scale*, manuscript.

The User Wants a Package the Catalog Does Not Offer

YOUR WEEKEND STAY

City center
2 nights · 2 guests
Breakfast included

Up to € 240

TARGET SID

$(12, 7, 5)$

Desired package—
not in the catalog

Hotel A
City center · 2 nights
€ 220
Breakfast not included
Add breakfast: € 40

NEARBY CATALOG SID
$(12, 7, 3)$
City center; room only

Hotel B
City center · 2 nights
€ 270
Breakfast included
€ 30 over budget

NEARBY CATALOG SID
$(12, 7, 9)$
City center; breakfast

Illustrative codes and neighbors; similarity is measured in the learned representation space.

The target describes a desired exposure; it need not resolve to an existing package.

Plan an Offer, Then Show the User a Concrete Package



Validated offers; no stacking. Choose the lowest guest payment.

€ 40 — **Breakfast voucher** — SELECTED
220 + 40 − 40 = **€ 220**

10% — **Room discount**
198 + 40 = **€ 238**

€ 10 — **Fixed discount** — Over € 240 budget
220 + 40 − 10 = **€ 250**

Hotel A

City center · 2 nights
Breakfast for two included
€ 220 total
View offer
2 guests; eligible breakfast voucher applied

Choose an executable offer first;
then generate its presentation.

Planning chooses a feasible way to realize the target, rather than only selecting the nearest item.

Research Agenda for Semantic Planning



Target Representation

What should the target encode?

Compare attributes, text and structured codes.



Grounded Actions

Which targets can we actually realize?

Test inventory and offer constraints.



Stakeholder Trade-offs

Whose objective determines the offer?

Compare user benefit, provider cost and platform value.



Evidence of Value

Does explicit planning add value?

Compare against direct and intent-conditioned recommendation.

An explicit target is useful only if it changes decisions in a measurable, justified way.

Open Questions for the Next Wave of Semantic IDs

Design

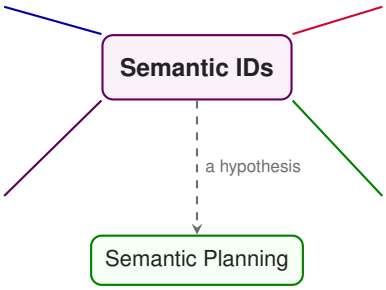
What should be shared?

Controlled comparisons

Evolution

What must survive updates?

Update cases



Evidence

What would establish the benefit?

Negative results

Distribution

Who benefits, and who is left behind?

Distributional audits

The next step is better questions and stronger evidence—not a single default ID design.

Summary

One Representation, an Entire Recommendation Pipeline

Information
Multimodal content
Collaborative signals

→

Semantic IDs
Learn reusable
discrete codes

→

Recommendation
Generate IDs
or retrieve / rank

→

Evidence
Quality, costs
and who benefits



A: (a_{12}, b_7, c_3) - - - - - - B: (a_{12}, b_7, c_9)



Applications to validate

Shared codes create opportunities.
Benefits depend on the task and
model.

Cold start
New and
sparse-history items

Cross-domain transfer
Connect different domains

Search–Rec unification
Shared item space
across tasks

The value of an ID depends on what it encodes, how it is consumed, and what it improves.

What Would Make an ID Design Worth Using?

Question	What to check
What is being shared?	Inspect code structure, similar and different items, and collisions. Identify the basis for sharing.
What actually improves?	Compare raw IDs, content vectors and SIDs under matched settings. Test overall quality, new items and cross-domain transfer.
What does it cost—and who benefits?	Measure storage, consumption or decoding, and updates. Check differences in user utility and provider exposure.

A meaningful code is a hypothesis; downstream evidence establishes its value.

What Should an ID Mean?



Meaning determines what can be shared.



Consumption determines what the model can use.



Evidence determines whether the design is useful.

What should be shared, and who benefits?

semantic-id-tutorial.github.io